Universidade de Brasília – UnB
Departamento de Ciência da Computação | Programação Concorrente

Relatório Técnico: Produtor-Consumidor com Buffer Limitado

Comunicação em Memória Compartilhada e Logger Assíncrono com POSIX Pthreads

Aluno: Gustavo Choueiri **Data:** Novembro de 2026

1 Descrição do Problema e Condições de Corrida

O objetivo deste trabalho é solucionar o problema clássico de comunicação e sincronização de processos através de uma região de memória compartilhada, eliminando condições de corrida por meio das primitivas da biblioteca POSIX Pthreads (`pthread_mutex_t`, `pthread_cond_t` e `sem_t`).

O cenário implementado consiste em um **buffer circular limitado** de capacidade $N = 5$ posições de números inteiros, acessado simultaneamente por:

- **3 Threads Produtoras:** geram itens sequenciais e os depositam no buffer compartilhado.
- **3 Threads Consumidoras:** retiram itens do buffer e simulam o seu processamento.

Sem mecanismos de controle de concorrência, o sistema estaria sujeito a quatro condições de corrida críticas:

1. **Inserção concorrente:** dois produtores calculam o mesmo índice de cauda e gravam na mesma posição, corrompendo e perdendo valores.
2. **Remoção concorrente:** dois consumidores leem a mesma posição de cabeça, duplicando o consumo de um item e inconsistenciando a contagem.
3. **Buffer Overflow:** um produtor tenta inserir dados com o buffer lotado ($quantidade = N$).
4. **Buffer Underflow:** um consumidor tenta retirar dados com o buffer vazio ($quantidade = 0$).

2 Solução Desenvolvida e Mecanismos de Sincronização

A solução foi organizada modularmente em arquivos C (módulos `buffer`, `produtor`, `consumidor`, `logger` e `main`), empregando as primitivas POSIX em duas frentes:

2.1 Buffer Compartilhado (Mutex e Variáveis de Condição)

O acesso ao buffer central é protegido com:

- `pthread_mutex_t` (`mutex_buffer`): assegura **exclusão mútua** estrita em operações de leitura, escrita e atualização de índices e contadores.
- `pthread_cond_t` (`nao_cheio`): bloqueia produtores se o buffer estiver cheio. O teste é feito via `while (quantidade == TAMANHO_BUFFER)` para tratar despertares espúrios.
- `pthread_cond_t` (`nao_vazio`): suspende consumidores se o buffer estiver vazio (`quantidade == 0`).

Ao inserir um item, o produtor sinaliza `nao_vazio`; ao retirar, o consumidor sinaliza `nao_cheio`.

2.2 Logger Assíncrono Desacoplado (Semáforos POSIX)

Para auditar as operações sem introduzir atrasos nas threads principais, implementou-se uma fila circular de logs independente sincronizada com **semáforos POSIX** (`sem_t`):

- `sem_espacos_log`: semáforo contador iniciado em 20 (capacidade da fila), controlando slots livres.
- `sem_itens_log`: semáforo contador iniciado em 0, representando mensagens pendentes de escrita.
- `sem_trywait()`: chamada **não-bloqueante** no envio de mensagens. Se a fila de logs estiver cheia, o evento é descartado sem jamais bloquear a execução dos produtores ou consumidores.

Primitiva POSIX	Componente	Papel na Sincronização
`pthread_mutex_t`	Buffer	Exclusão mútua nas regiões críticas de inserção e remoção.
`pthread_cond_t`	Buffer	Bloqueio/sinalização de buffer cheio e vazio (`nao_cheio`, `nao_vazio`).
`sem_t`	Logger	Controle de fluxo produtor-consumidor da fila assíncrona de eventos.

3 Resultados e Validação Experimental

O programa foi compilado com `gcc -Wall -Wextra -pthread` via `Makefile` e executado com 3 produtores e 3 consumidores, totalizando 12 itens transacionados (4 por produtor).

A saída registrada pelo logger assíncrono comprova a corretude e a sincronização temporal:

```
[ 158.6ms] [P3] Inseriu:  1 | Buf:  1/5        [ 840.7ms] [C1] Retirou:  5 | Buf:  4/5
[ 212.4ms] [P2] Inseriu:  2 | Buf:  2/5        [ 840.8ms] [P3] Inseriu:  10 | Buf:  5/5
[ 241.5ms] [P1] Inseriu:  3 | Buf:  3/5        [ 1046.8ms] [P2] Cheio.  Aguarda...
[ 281.4ms] [C3] Retirou:  1 | Buf:  2/5        [ 1072.7ms] [C3] Retirou:  7 | Buf:  4/5
[ 620.0ms] [P1] Inseriu:  8 | Buf:  5/5        [ 1072.8ms] [P2] Inseriu:  12 | Buf:  5/5
[ 759.1ms] [P3] Cheio.  Aguarda...             [ 1745.4ms] [C2] Retirou:  12 | Buf:  0/5
```

=== Execução concluída: 12 itens produzidos e 12 consumidos ===

Comprovações experimentais:

1. **Bloqueio em buffer cheio:** Em 759,1 ms e 1046,8 ms, os produtores detectaram ocupação máxima (5/5) e bloquearam em `nao_cheio`, impedindo qualquer transbordamento de dados.
2. **Retomada imediata:** Assim que um consumidor liberou vaga (em 840,7 ms e 1072,7 ms), o produtor suspenso acordou prontamente e efetuou a inserção com sucesso.
3. **Integridade e término limpo:** Todos os 12 itens foram consumidos sem perda ou duplicação. O buffer encerrou vazio (0/5), e a aplicação finalizou sem *deadlock* ou vazamento de recursos.

4 Conclusão

A implementação atendeu a todas as especificações do trabalho de forma simples, objetiva e eficaz. O uso coordenado de `pthread_mutex_t` e `pthread_cond_t` eliminou por completo as condições de corrida no acesso à memória compartilhada do buffer circular. Paralelamente, o emprego de semáforos POSIX (`sem_t`) no logger assíncrono demonstrou como primitivas distintas podem ser combinadas harmoniosamente em uma arquitetura concorrente de alto desempenho e confiabilidade.